

Отчет по выполнению внешнего курса(3 этап)

Мартынова Милана Александровна

Содержание

1. Цель работы	6
2. Задание	7
3. Теоретическое введение	8
4. Выполнение	9
5. Выводы	28
Список литературы	29

Список иллюстраций

1	Как выйти из vim	9
2	Перемещение по словам	10
3	Клавиши для редактирования	10
4	Команда для первой замены в vim	11
5	Утверждения про режим Visual	11
6	История команд bash	12
7	Абсолютный путь созданного файла touch file1.txt	12
8	Допустимые имена переменных в bash	13
9	Скрипт на вывод аргументов	13
10	Всегда истинные условия if	14
11	Логика ветвления скрипта	14
12	Скрипт: число студентов	15
13	Перемещение по словам	15
14	Скрипт: возрастные группы	16
15	Увеличение переменной в bash	16
16	Перемещение по словам	17
17	Работа с кодом возврата	17
18	бласть видимости переменных в bash	18
19	Перемещение по словам	18
20	Скрипт “Калькулятор”	19
21	Разница find -name и -iname	19
22	Понимание find -path	20
23	Глубина поиска mindepth / maxdepth	21
24	Контекст grep	21
25	Регулярные выражения в grep -E	22
26	Работа sed без ключа -n	22
27	регулярное выражение sed для замены аббревиатур	23
28	Опция gnuplot против закрытия окна графиков	23
29	Название построенного ряда данных и точки на графике	24
30	Командой set xtics задать подписи	24
31	Перемещение по словам	25
32	Права доступа rwxrwx-r— файлу file.txt	25
33	Команда чтобы создать файл внутри dir	26
34	Характеристики файла с использованием wc	26
35	Команда для вывода места на диске	27
36	Команда создания 3 поддиректория	27

1	Сертификат	28
---	----------------------	----

Список таблиц

1. Цель работы

Изучение основ операционной системы Linux, освоении работы в командной строке, получении навыков администрирования сервера и написания скриптов на языке bash через прохождение внешнего образовательного курса.

2. Задание

- Изучение теории курса.
- Выполнить все тестовые задания, подтвердив понимания теории.
- Выполнить все практические/интерактивные задания в терминале.
- Написать отчет.

3. Теоретическое введение

Linux — это свободное семейство операционных систем с открытым кодом на базе ядра Linux. В отличие от Windows, он не требует оплаты, даёт полную свободу настроек, более защищён и работает везде: от серверов и суперкомпьютеров до Android-смартфонов и умных устройств.

4. Выполнение

Задание 3.1 Формулировка: Как выйти из vim? Пояснение по выбору ответа: Нужно перейти в командный режим и ввести команду выхода без сохранения: нажать :, затем q, затем Enter. (рис. 1)

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

✓ Верно. Так держаты!

- ☐ ":", затем "q"
- ☒ ":", затем "q", затем "Enter"
- ☐ "Esc"
- ☐ "Ctrl", затем "x"
- ☐ "q", затем "Enter"

Следующий шаг Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 47 488 учащихся
Из всех попыток 67% верных

Рис. 1: Как выйти из vim

Задание 3.2 Формулировка: Перемещение по словам (word / WORD). Пояснение по выбору ответа: Строчные клавиши (w) учитывают спецсимволы как разделители, а заглавные (W) — только пробелы. Были подсчитаны слова по этим правилам. (рис. 2)

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению `w`, нужно открыть `vim` и ввести команду `:help w`. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про `e` и про `b` и, самое главное, про `word` и `WORD`. Кроме того, можно вызвать сразу справку по термину `word` при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Выберите все подходящие ответы из списка

☒ Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☒ В этой строке 9 "слов" (word)
☐ Чтобы попасть в конец строки, нужно совершить больше нажатий на W, чем на w
☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
☐ В этой строке 9 "больших слов" (WORD)
☐ В этой строке 12 "слов" (word)
☒ После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w

Следующий шаг
 Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 36 625 учащихся
 Из всех попыток 19% верных

Рис. 2: Перемещение по словам

Задание 3.3 Формулировка: Клавиши для редактирования one two three four five. Пояснение по выбору ответа: Команда `d2w` (удалить два слова), `$` (в конец), `b` (назад на слово), `i` (режим вставки) four four. (рис. 3)

Какие(ой) из предложенных ниже наборов нажатий клавиш выполнят такое редактирование? В этих наборах нажатие на клавишу `Esc` обозначается как `<Esc>` (т.е. знаки `"<"` и `">"` не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе `vim`, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☒ `d2w$bifour four<Esc>`
☐ `d2wwywpP`
☐ `d2dwywPp`
☐ `x2wwywpP`
☒ `d2wwywpP`
☒ `dditthree four four five<Esc>`

Следующий шаг
 Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 34 162 учащихся
 Из всех попыток 16% верных

Рис. 3: Клавиши для редактирования

Задание 3.4 Формулировка: Команда для первой замены в `vim`. Пояснение по выполнению: Чтобы заменить только первое вхождение слова в каждой строке, глобальный флаг `g` не ставится. Команда: `:%s/Windows/Linux/`.

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. включая ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

✓ Верно.

Следующий шаг
Решить снова

Вы получили: 2 балла
[Мои решения](#)

Верно решили 35 989 учащихся
Из всех попыток 61% верных

Рис. 4: Команда для первой замены в vim

Задание 3.5 Формулировка: Утверждения про режим Visual. Пояснение по выбору ответа: В визуальном режиме можно перемещаться (w, \$), копировать (y), удалять (d). Индикатор режима — надпись — **VISUAL** —.

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в `vimtutor`, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить `vimtutor` целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

✓ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ Режим выделения открывается при помощи команды `:visual`
☐ Чтобы выйти из режима выделения, нужно ввести `:q`
☒ Режим выделения открывается из нормального режима по нажатию `"v"`
☐ Режим выделения открывается из любого другого режима по нажатию `"v"`
☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись — **VISUAL** — (или — **ВИЗУАЛЬНЫЙ РЕЖИМ** —)
☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)

Следующий шаг
Решить снова

Рис. 5: Утверждения про режим Visual

Задание 3.6 Формулировка: История команд bash. Пояснение по выбору ответа: Каждая запущенная подболочка хранит свою локальную историю команд до завершения сеанса (Набор C). (рис. 6)

3.2 Скрипты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попробуете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

☒ Всё правильно.

☐ Из наборов B и C

☐ Только из набора A

☒ Только из набора C

☐ Из наборов A и C

☐ Только из набора B

Следующий шаг Решить снова

Вы получили: 1 балл
Мои решения

Верно решил 44 341 учащийся
Из всех попыток 63% верных

Рис. 6: История команд bash

Задание 3.7 Формулировка: Абсолютный путь созданного файла `touch file1.txt`. Пояснение по выбору ответа: Команда была выполнена после `cd /home/bi/`, значит путь будет `/home/bi/file1.txt`. Дальнейшие `cd` на файл уже не влияют. (рис. 7)

3.2 Скрипты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть абсолютный путь до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

☒ Хорошая работа.

☐ `/home/Desktop/file1.txt`

☒ `/home/bi/file1.txt`

☐ Никкак (файла `file1.txt` не будет существовать после завершения работы скрипта)

☐ `/home/bi/Documents/file1.txt`

Следующий шаг Решить снова

Вы получили: 1 балл
Мои решения

Верно решил 45 607 учащийся
Из всех попыток 63% верных

Рис. 7: Абсолютный путь созданного файла `touch file1.txt`

Задание 3.8 Формулировка: Допустимые имена переменных в `bash`. Пояснение по выбору ответа: Разрешены буквы, цифры и символ `_`. Имя не может начинаться с цифры и не должно содержать спецсимволов (вроде `@`). (рис. 8)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

✓ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ var.i.able
☐ var i able
☒ VARiable
☒ variable
☒ variable123
☒ variable_123
☒ _variable

Следующий шаг
Решить снова

Рис. 8: Допустимые имена переменных в bash

Задание 3.9 Формулировка: Скрипт на вывод аргументов. Пояснение по выполнению: Написан скрипт, использующий встроенные переменные позиционных аргументов \$1 и \$2. (рис. 9)

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](#).

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание на наши [рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Абсолютно точно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 var1=$1
3 var2=$2
4 echo "Arguments are: \${1=$var1} \${2=$var2}"
5
6
```

Рис. 9: Скрипт на вывод аргументов

Задание 3.10 Формулировка: Всегда истинные условия if. Пояснение по выбору ответа: 5 -ge 5 (числовое сравнение), -z "" (пустая строка), -n \$0 (имя скрипта непустое). (рис.

10)

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

✓ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ 5 -ge 5
- ☐ -n \$1
- ☐ \$# -gt 0
- ☐ \$var1 == \$var2 && \$var1 != \$var2
- ☒ -z ""
- ☒ -n \$0

Следующий шаг Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 33 918 учащихся
Из всех попыток 16% верных

Рис. 10: Всегда истинные условия if

Задание 3.11 Формулировка: Логика ветвления скрипта. Пояснение по выбору ответа: Согласно коду, в обоих случаях (`var=3`, `var=5`) не выполняется ни одно из условий `if/elif`, поэтому срабатывает блок `else` (выводит “four”). (рис. 11)

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

✓ Всё правильно.

- ☐ Сначала two, потом four
- ☒ Сначала four, потом four
- ☐ Сначала two, потом one
- ☐ Сначала four, потом one

Следующий шаг Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 37 336 учащихся
Из всех попыток 62% верных

Рис. 11: Логика ветвления скрипта

Задание 3.12 Формулировка: Скрипт: число студентов. Пояснение по выполнению: Реализовано ветвление на `bash` с проверкой входа (`-eq 0`, `-eq 1`, `-ge 2 && -le 4`) для правильного склонения числительных. (рис. 12)

примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно **проверять вход** (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать **не нужно!**

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Правильно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 if [[ $1 -eq 1 ]]; then
2   echo "1 student"
3 elif [[ $1 -gt 1 && $1 -le 4 ]]; then
4   echo "1 students"
5 elif [[ $1 -ge 5 ]]; then
6   echo "A lot of students"
7 else
8   echo "No students"
9 fi
10
```

Рис. 12: Скрипт: число студентов

Задание 3.13 Формулировка: Сколько раз выведет start/finish в цикле? Пояснение по выбору ответа: Цикл выполняется 5 раз ("start"). На одной из итераций срабатывает continue, обрывая текущий шаг, поэтому "finish" выводится 4 раза. (рис. 13)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c,d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово "start", а сколько раз слово "finish"?

Выберите один вариант из списка

✓ Всё правильно.

- ☐ 3 раза "start" и ни разу "finish"
- ☒ 5 раз "start" и 4 раза "finish"
- ☐ 3 раза "start" и 2 раза "finish"
- ☐ 5 раз "start" и 2 раза "finish"

Следующий шаг

Решить снова

Рис. 13: Перемещение по словам

Задание 3.14 Формулировка: Скрипт: возрастные группы. Пояснение по выполнению: Написан while true цикл с командой read. Логика if/elif распределяет пользователя в груп-

пу child/youth/adult и завершает цикл при вводе 0 или пустоты. (рис. 14)

```

1 child=16
2 adult=25
3 stdout=0
4
5 while [[ $stdout != 1 ]]
6 do
7     echo "enter your name: "
8     read name
9     if [[ (-z $name) || ($name = 0) ]]; then
10        echo "bye"
11        stdout=1
12    elif [[ -n $name ]]; then
13        while [[ $stdout != 1 ]]; do
14            echo "enter your age: "
15            read age
16            if [[ ($age -eq 0) || (-z $age) ]]; then
17                echo "bye"
18                stdout=1
19            elif [[ $age -le $child ]]; then
20                echo "$name, your group is child"
21            elif [[ $age -gt $adult ]]; then
22                echo "$name, your group is adult"; else
23                if [[ ($age -ge 17) && ($age -le 25) ]]; then
24                    echo "$name, your group is youth"; fi
25                fi; break
26            done ; fi
27        done

```

Рис. 14: Скрипт: возрастные группы

Задание 3.15 Формулировка: Увеличение переменной в bash. Пояснение по выбору ответа: Последние 3 предложенных варианта использования команды let (с кавычками, без, с +=) синтаксически корректно отрабатывают сложение в bash, let допускает пробелы внутри выражения, но не после имени переменной перед = и не после +=. (рис. 15)

Выберите все подходящие варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ \$ тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания \$ в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и пробелы, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

☒ Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ let a = a + b

☐ a+= b

☒ let a=a+b

☒ let "a= a+ b"

☒ let a= a+ b

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл Верно решили 32 789 учащихся
[Мои решения](#) Из всех попыток 20% верных

Рис. 15: Увеличение переменной в bash

Задание 3.16 Формулировка: Что выведет echo “pwd”. Пояснение по выбору ответа: Обратные кавычки вызывают команду внутри subshell. Результат “pwd”(путь/home/bi) подставляется в команду echo (рис. 16)

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [programs.sh](#).

Пусть вы находитесь в директории /home/bi/Documents/ и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
echo "`pwd`"
```

Что в этом случае выведет команда echo на экран?

Выберите один вариант из списка

☒ Здорово, всё верно.

☐ `pwd`

☐ /home/bi/Documents

☐ Код возврата команды pwd (0 в случае успешного выполнения и не 0 в случае ошибок)

☐ pwd

☒ /home/bi

Следующий шаг Решить снова

Вы получили: 1 балл
Мои решения

Верно решили 35 282 учащихся
Из всех попыток 51% верных

Рис. 16: Перемещение по словам

Задание 3.17 Формулировка: Работа с кодом возврата (exit code). Пояснение по выбору ответа: Переменная \$? хранит код завершения последней программы. Правильная конструкция: Запустить program, затем проверить if [[\$? -eq 0]]. (рис. 17)

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции if при помощи if `program options arguments` (действия внутри if выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в stdout, то в проверку if поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой bash-скрипт с использованием, например, if `pwd`.

Однако как быть, если хочется всё-таки запустить программу program, которая пишет что-то в stdout и потом выполнить какие-то действия если ее код возврата равен 0? Выберите все верные утверждения или правильно работающие конструкции if.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно косая кавычка ('), а не обычная (") или двойная (").

Выберите все подходящие ответы из списка

☒ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в комментариях, отвечая на их вопросы, или сравнить своё решение с другими на форуме решений.

☒ Сначала запустить program, затем if [[\$? -eq 0]]

☐ Сначала var="program", затем if [[\$var -eq 0]]

☐ if [[`program` -eq 0]]

☒ if `program` > some_file.txt

☐ Ничего сделать нельзя

Рис. 17: Работа с кодом возврата

Задание 3.18 Формулировка: Область видимости переменных в `bash`. Пояснение по выбору ответа: Ключевое слово `local` оставляет `c1` внутри функции. Переменная `c2` глобальна, она аккумулируется в цикле, выдавая 55 и 110. (рис. 18)

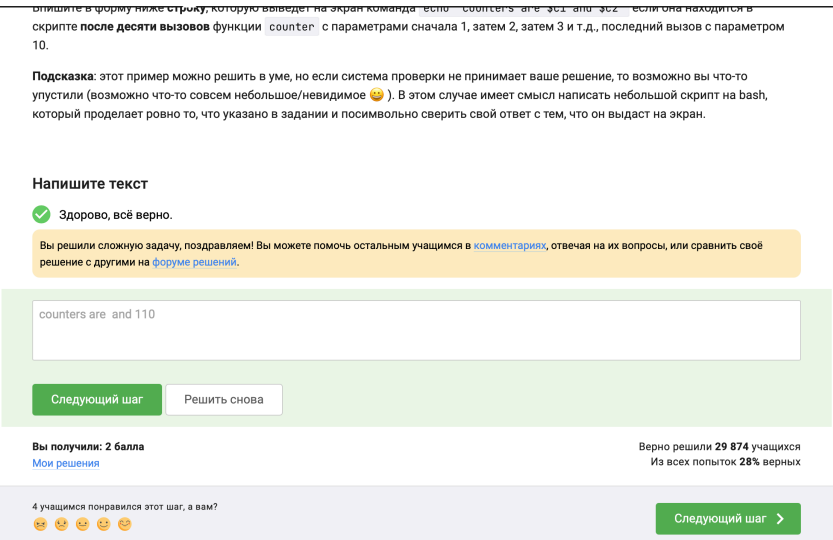


Рис. 18: бласть видимости переменных в `bash`

Задание 3.19 Формулировка: Скрипт НОД (алгоритм Евклида). Пояснение по выполнению: Написана рекурсивная функция `gcd`, которая вычитает меньшее число из большего, пока они не сравняются. (рис. 19)



Рис. 19: Перемещение по словам

Задание 3.20 Формулировка: Скрипт “Калькулятор”. Пояснение по выполнению: Использована конструкция case для обработки знаков арифметических операций и перехвата ошибок при неверном вводе. (рис. 20)

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```

1 #!/bin/bash
2 while [[ True ]]
3 do
4     read birinchi amal ikkinchi
5     if [[ $birinchi == "exit" ]]
6     then
7         echo "bye"
8         break
9     elif [[ "$birinchi" =~ ^[0-9]+$ && "$ikkinchi" =~ ^[0-9]+$ ]]
10    then
11        echo "error"
12        break
13    else
14        case $amal in
15            "+") let "result = birinchi + ikkinchi";;
16            "-") let "result = birinchi - ikkinchi";;
17            "/" ) let "result = birinchi / ikkinchi";;
18            "*" ) let "result = birinchi * ikkinchi";;
19            "%" ) let "result = birinchi % ikkinchi";;
20            "**") let "result = birinchi ** ikkinchi";;
21            *) echo "error" ; break ;;
22        esac
23        echo "$result"
24    fi
25 done

```

Следующий шаг Решить снова

Вы получили: 5 баллов Верно решили 26 748 учащихся

Рис. 20: Скрипт “Калькулятор”

Задание 3.21 Формулировка: Разница find -name и -iname. Пояснение по выбору ответа: Флаг -iname игнорирует регистр символов. Поэтому он найдет STARS.txt и Star_Wars.avi, в то время как строгий -name их пропустит. (рис. 21)

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Пусть в директории /home/bi лежат файлы Star_Wars.avi, star_trek_OST.mp3, STARS.txt, stardust.mpeg, Eddard_Stark_biography.txt.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*"` ?

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

☐ stardust.mpeg

☒ Star_Wars.avi

☐ star_trek_OST.mp3

☐ Eddard_Stark_biography.txt

☒ STARS.txt

Следующий шаг Решить снова

Вы получили: 1 балл Моё решение Верно решили 30 944 учащихся Из всех попыток 40% верных

Рис. 21: Разница find -name и -iname

Задание 3.22 Формулировка: Понимание `find -path`. Пояснение по выбору ответа: `path` ищет совпадения по всему абсолютному пути, а `name` — только по имени файла. Иногда результаты могут совпадать или `name` найдет больше (если паттерн не включает слеша). (рис. 22)

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте все верные утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✓ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ Если заменить в команде поиска `-name` на `-path`, то результат поиска иногда может остаться таким же
- ☐ Опции `-path` и `-name` всегда работают одинаково
- ☐ Если заменить в команде поиска `-name` на `-path`, то результат поиска всегда останется неизменным
- ☒ В некоторых случаях `find` с `-name` найдет больше файлов, чем `find` с таким же запросом, но с `-path`

Следующий шаг Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 2
Из всех попыток

Рис. 22: Понимание `find -path`

Задание 3.23 Формулировка: Глубина поиска `mindepth` / `maxdepth`. Пояснение по выбору ответа: Установленные лимиты (глубина 2 и 3) отсекают файл `file3`, который лежит на глубине 4. Остальные будут найдены. (рис. 23)

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```

/home/bi/
├── dir1
│   ├── file1
│   ├── dir2
│   │   ├── file2
│   │   └── dir3
│   │       └── file3
└── file3
  
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file"` ?

Выберите один вариант из списка

☒ Всё правильно.

☐ Только `file3`

☐ Все кроме `file1`

☒ Все кроме `file3`

☐ Все три файла

☐ Ни один файл найден не будет

[Следующий шаг](#) [Решить снова](#)

Рис. 23: Глубина поиска `mindepth` / `maxdepth`

Задание 3.24 Формулировка: Контекст `grep` (`-A`, `-B`, `-C`). Пояснение по выбору ответа: Поскольку слово “word” есть в каждой строке файла, ключи контекста (до, после) не добавляют новых строк. Размер файла во всех случаях одинаков. (рис. 24)

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в каждой строке есть слово “word”. Если вы выполните на этом файле команды:

```

grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
  
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

☒ Верно.

☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`

☐ `grep -A 1 "word" file.txt > results.txt`

☐ Все, кроме `grep "word" file.txt > results.txt`

☐ `grep -C 1 "word" file.txt > results.txt`

☒ `results.txt` будет одинакового размера во всех случаях

[Следующий шаг](#) [Решить снова](#)

Рис. 24: Контекст `grep`

Задание 3.25 Формулировка: Регулярные выражения в `grep` `-E`. Пояснение по выбору ответа: потому что `X` подходит под `[xkLXL]`, а `buntu` — в конце строки. (рис. 25)

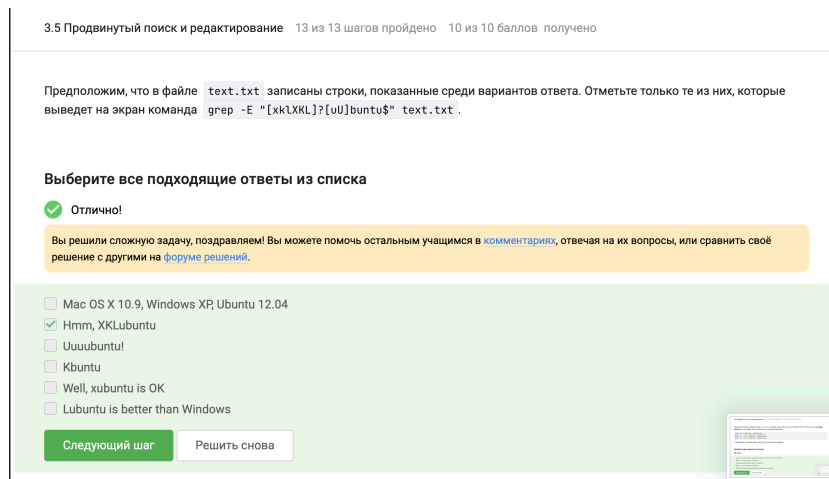


Рис. 25: Регулярные выражения в `grep -E`

Задание 3.26 Формулировка: Работа `sed` без ключа `-n`. Пояснение по выбору ответа: Без “тихого режима” `sed` выводит паттерн (из-за флага `/p`) и одновременно дублирует стандартный вывод (по умолчанию), выводя строки дважды. (рис. 26)

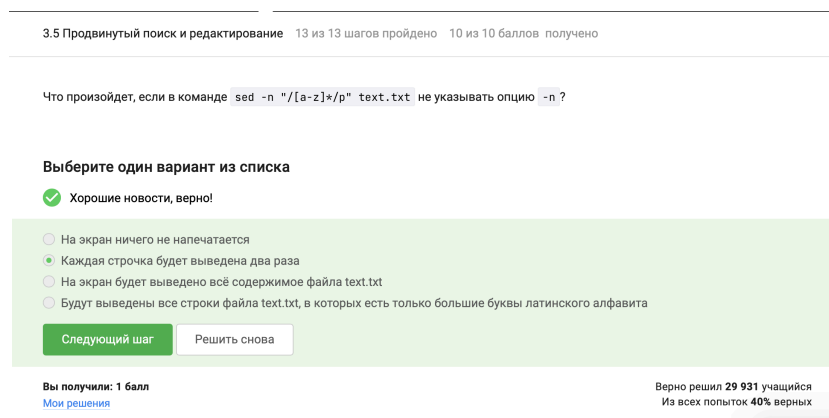


Рис. 26: Работа `sed` без ключа `-n`

Задание 3.27 Формулировка: Регулярное выражение `sed` для замены аббревиатур. Пояснение по выполнению: Запрос `sed 's/[A-Z]{2,} / abbreviation /g' input.txt > edited.txt` находит слова из двух и более заглавных латинских букв, окруженные пробелами. (рис. 27)

можно же ввести `rm -rf /*`!) Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её смысловую часть** (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах. К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная "несовместимость" заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. `*`, `+`, `?` и `{}`). Однако эту "несовместимость" легко исправить указав при помощи "(" и ")" какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве "a") нужно записать как `(a+)?` (при этом запись `(a)+?`, конечно же, не поможет).

Напишите текст

✓ Абсолютно точно.

`sed 's/[A-Z]\{2,\} /abbreviation /g' input.txt > edited.txt`

Следующий шаг

Решить снова

Вы получили: 3 балла
[Мои решения](#)

Верно решили 25 926 учащихся
Из всех попыток 41% верных

Рис. 27: егулярное выражение `sed` для замены аббревиатур

Задание 3.28 Формулировка: Опция `gnuplot` против закрытия окна графиков. Пояснение по выбору ответа: Ключ `-p` или `-persist` заставляет графическое окно X11 оставаться открытым после завершения скрипта рисования. (рис. 28)

3.6 Строим графики в `gnuplot` 10 из 10 шагов пройдено 7 из 7 баллов получено

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

✓ Отличное решение!

- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!
- ☐ `-raise`
- ☒ `-p, -persist`
- ☐ Такой опции не существует

Следующий шаг

Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 2
Из всех попыток

Рис. 28: Опция `gnuplot` против закрытия окна графиков

Задание 3.29 Формулировка: Название построенного ряда данных и точки на графике. Пояснение по выбору ответа: `columnhead` берёт заголовок из первой строки второго столбца, а саму строку не рисует → 9 точек. (рис. 29)

Предположим у вас есть файл 'data.csv' с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете gnuplot и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один вариант из списка

☒ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Название – первое значение из первого столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название "data.csv using 1:2", нарисовано 10 точек
- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☐ Название "попате", нарисовано 10 точек

Следующий шаг

Решить снова



Рис. 29: Название построенного ряда данных и точки на графике

Задание 3.30 Формулировка: Командой `set xtics` задать подписи. Пояснение по выбору ответа: Конкатенация строк через `.` формирует нужную подпись, затем указывается координата. Пробел между `value` и `x1` не нужен — `.x1` добавляет значение сразу. (рис. 30)

точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно**!

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин **конкатенация**, который важен для выполнения данного задания. Под **конкатенацией** обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст

☒ Здорово, всё верно.

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)
```

Следующий шаг

Решить снова

Вы получили: 2 балла
[Мои решения](#)

Верно решили 22 768 учащихся
Из всех попыток 54% верных

Рис. 30: Командой `set xtics` задать подписи

Задание 3.31 Формулировка: Изменений инструкций в файле `move.rot`. Пояснение по выбору ответа: `zrot+350` вместо `+10` даёт вращение назад и быстрее, а `-x2-y2` зеркалит

график. (рис. 31)

Файле `move.rot` (т.е. **добавить** и **удалить** инструкции **нельзя**), таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу `gnuplot` и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в `gnuplot` работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст

✓ Хорошие новости, верно!

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Следующий шаг Решить снова

Рис. 31: Перемещение по словам

Задание 3.33 Формулировка: Права доступа `rwXrW-r--` файлу `file.txt`. Пояснение по выбору ответа: `a+wx` → все получают `wx`, затем `o-wx` и `g-x` убирают лишнее, получая `rwXrW-r--`. (рис. 32)

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Какая команда(ы) установят файлу `file.txt` права доступа `rwXrW-r--`, если изначально у него были права `r--r--r--`. Укажите **все верные варианты ответа!**

Примечание: запись вида команда1; команда2; команда3 означает, что в терминале последовательно выполнялись все три команды (сначала команда1, затем команда2 и, наконец, команда3).

Выберите все подходящие ответы из списка

✓ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ `chmod 467 file.txt`
☐ `chmod u-wx file.txt; chmod g-w file.txt`
☐ `chmod 777 file.txt`
☒ `chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt`
☐ `chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt`
☐ `chmod rwXrW-r-- file.txt`

Следующий шаг Решить снова

Рис. 32: Права доступа `rwXrW-r--` файлу `file.txt`

Задача 3.33 Формулировка: Команда чтобы создать файл внутри `dir`? Пояснение по выбору ответа: `sudo chmod a+w dir` даёт право на запись всем, включая `user`. (рис. 33)

Выберите все подходящие ответы из списка

✓ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ chmod o+w dir
- ☐ chown user:group dir
- ☐ sudo chmod g+w dir
- ☐ sudo chmod o+x dir
- ☐ sudo chown :group dir
- ☒ sudo chmod a+w dir

Следующий шаг Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 22 855 учащихся
Из всех попыток 17% верных

Рис. 33: Команда чтобы создать файл внутри dir

Задача 3.34 Формулировка: Характеристики файла с использованием wc. Пояснение по выбору ответа: wc -с, -L, -w, -l, -m — считает всё перечисленное. (рис. 34)

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

✓ Хорошие новости, верно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Размер файла в байтах
- ☒ Длину самой длинной строки
- ☒ Количество слов
- ☒ Количество строк
- ☒ Количество символов

Следующий шаг Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 2
Из всех попыток 17% верных

Рис. 34: Характеристики файла с использованием wc

Задача 3.35 Формулировка: Команда для вывода места на диске. Пояснение по выбору ответа: du -hs — размер текущей папки в удобном формате. (рис. 35)

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо 2948 байт надо вывести 2.9К) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 880 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4М (также на экране может быть выведен еще и символ "", обозначающий, что это размер именно текущей директории).

Напишите текст

✓ Верно. Так держать!

du -h -s

Следующий шаг

Решить снова

Вы получили: 2 балла

Мои решения

Верно решили 4

Из всех попыток 59% верных

Рис. 35: Команда для вывода места на диске

Задача 3.36 Формулировка: Команда создания 3 поддиректория. Пояснение по выбору ответа: `mkdir dir{1..3}` — фигурные скобки генерируют `dir1 dir2 dir3`, создавая три папки одной командой. (рис. 36)

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✓ Верно.

mkdir dir{1..3}

Следующий шаг

Решить снова

Вы получили: 2 балла

Мои решения

Верно решили 4

Из всех попыток 59% верных

Рис. 36: Команда создания 3 поддиректория

5. Выводы

В ходе выполнения заданий внешнего курса были изучены и закреплены на практике базовые принципы взаимодействия с POSIX-совместимыми системами.

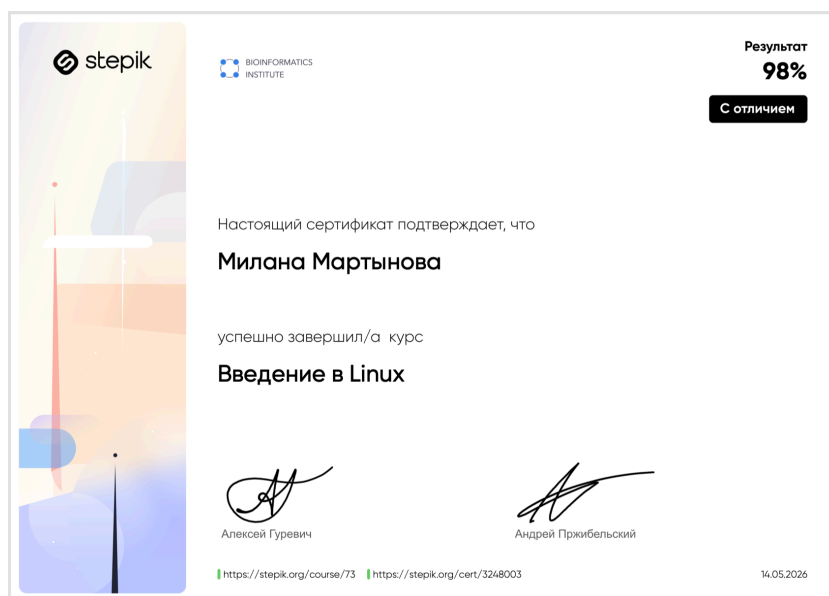


Рис. 1: Сертификат

Список литературы